

One-to-many Object Relationships Lab (CodeGrade)



<https://github.com/learn-co-curriculum/p3-oo-one-to-many-lab>  <https://github.com/learn-co-curriculum/p3-oo-one-to-many-lab/issues/new>

Learning Goals

- Create a one-to-many relationship.
-

Key Vocab

- **Class**: a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
 - **Object**: the more common name for an instance. The two can usually be used interchangeably.
 - **Object-Oriented Programming**: programming that is oriented around data (made mobile and changeable in **objects**) rather than functionality. Python is an object-oriented programming language.
 - **Function**: a series of steps that create, transform, and move data.
 - **Method**: a function that is defined inside of a class.
-

Introduction

In this lab we will implement a one-to-many relationship between a **Owner** and **Pet** .

Instructions

This is a **test-driven lab**. Run `pipenv install` to create your virtual environment and `pipenv shell` to enter the virtual environment. Then run `pytest -x` to run your tests. Use these instructions and `pytest` 's error messages to complete your work in the `lib/` folder.

Instructions:

- Define an `Owner` class and pass into the constructor a `name` argument.
- Define a `Pet` and pass into the constructor its `name` , `pet_type` and an optional `owner` .
- Define a class variable `PET_TYPES = ["dog", "cat", "rodent", "bird", "reptile", "exotic"]` and validate that the `pet_type` is one of those types in the `__init__` method.
 - `raise Exception` if this check fails.
- Define a class variable `all` that stores all instances of the `Pet` class.
- In the `Owner` class write a method called `pets(self)` that returns a full list of the owner's pets.
- In the `Owner` class write a method called `add_pet(self, pet)` that checks that the the pet is of type `Pet` and adds the owner to the pet.
- In the `Owner` class write a method called `get_sorted_pets(self)` returns a sorted list of pets by their names.
- `Owner` and `Pet` should use `isinstance` to check types whenever instances are passed into methods.
 - `raise Exception` if these checks fail.

Resources

- [Python classes](https://docs.python.org/3/tutorial/classes.html)  (<https://docs.python.org/3/tutorial/classes.html>)

This tool needs to be loaded in a new browser window

Load One-to-many Object Relationships Lab (CodeGrade) in a new window